



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A
MINOR PROJECT PROPOSAL
ON
**EDUMENTX: LOCATION BASED TUTOR DISCOVERY
PLATFORM**

SUBMITTED BY:

APOORVA PAUDEL (NCE080BCT006)

ASIM POUDEL (NCE080BCT008)

DIBYA SHAKYA (NCE080BCT013)

SUHAN KHADKA (NCE080BCT043)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

FEBRUARY, 2026

CONTENTS

List of Figures	iii
List of Tables	iv
List of Abbreviations	vii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope	3
2 Literature Review	5
2.1 Related Work	5
2.2 Related Theory	6
2.2.1 Location-Based Services	6
2.2.2 Haversine Distance Formula	6
2.2.3 Geofencing and Radius-Based Filtering	7
2.2.4 K-Nearest Neighbors Algorithm	7
2.2.5 Retrieval-Augmented Generation	8
2.2.6 React Native and Expo	8
2.2.7 Firebase Platform and Supabase Storage	8
2.2.8 OpenStreetMap and Open Geocoders	9
3 Proposed Methodology	10
3.1 Software Development Model	10
3.2 Feasibility Study	10
3.2.1 Technical Feasibility	11
3.2.2 Economic Feasibility	11
3.2.3 Operational Feasibility	12
3.2.4 Legal and Ethical Feasibility	12

3.2.5	Scalability	13
3.3	Requirement Analysis	13
3.3.1	Functional Requirements	13
3.3.2	Non-Functional Requirements	15
3.4	Proposed System Design	16
3.4.1	System Architecture	16
3.4.2	Use Case Diagram	16
3.4.3	Data Flow Diagram	17
3.4.4	RAG Chatbot Workflow	18
3.5	Implementation Details	19
3.5.1	Frontend Development	19
3.5.2	Backend Development	19
3.5.3	Testing Strategy	20
4	Task Accomplished	21
4.1	Stage 0 — Project Setup and Documentation Scaffolding	21
4.2	Stage 1 — Design System: “Night & Sand” Premium Palette	21
4.3	Stage 1.5 — NativeWind Migration (Tamagui Removal)	22
4.4	Stage 2 — Native Firebase Auth (RNFirestore Pivot)	22
4.5	Stage 2b — Onboarding and Profile Subcollection Writes	22
4.6	Stage 3 — Dashboards and Live Firestore Reads	23
4.7	Stage A — Auth UX Polish, Bottom Nav, and Parity Foundations	23
4.8	Stage A.5 — Tutor Profile Parity, Avatar Persistence, and Logout	24
4.9	Stage B — 3D Onboarding Polish	25
4.10	Verification Summary	25
5	Tasks Remaining	26
5.1	Stage C — Live Tutor Query and KNN Ranking	26
5.2	Stage D — RAG Chatbot End-to-End	26
5.3	Stage E — Map Search and Service-Area Visualization	27
5.4	Stage F — Verification Queue and Blue Tick Pipeline	27
5.5	Stage G — Enrollment Request and Response Flow	27
5.6	Stage H — Rating and Review System	28

5.7	Stage I — Admin Statistics, User Management, and Polish	28
5.8	Stage J — Final Hardening, Performance Pass, and Demo	28
	References	28

LIST OF FIGURES

3.1 Phases of Iterative-Incremental Model 10

3.2 System Architecture of EdumentX 16

3.3 Use Case Diagram 17

3.4 Level 0 Data Flow Diagram 17

3.5 Level 1 Data Flow Diagram 18

3.6 RAG-Based AI Chatbot Workflow 18

LIST OF TABLES

2.1 KNN Feature Weights for Tutor Recommendation 8

3.1 Economic Feasibility Estimation 12

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ASCILITE	Australasian Society for Computers in Learning in Tertiary Education
BaaS	Backend-as-a-Service
CBS	Central Bureau of Statistics (Nepal)
DFD	Data Flow Diagram
ESP	Electronically Stored Personal data
GBV	Geospatial Bounding Value
GPS	Global Positioning System
GUI	Graphical User Interface
HTTP(S)	Hypertext Transfer Protocol (Secure)
IDE	Integrated Development Environment
IOE	Institute of Engineering
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
LBS	Location-Based Service
LLM	Large Language Model
MFA	Multi-Factor Authentication
MOOC	Massive Open Online Course
NCE	National College of Engineering
NLP	Natural Language Processing
NoSQL	Not only SQL
OTP	One-Time Password
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SEE	Secondary Education Examination (Grade 10, Nepal)
TU	Tribhuvan University
UAT	User Acceptance Testing
UI	User Interface
UX	User Experience

1. INTRODUCTION

1.1 Background

Education is one of the most important foundations for personal growth and social progress. Around the world, families invest heavily in their children's education, recognizing that formal schooling alone may not always be enough. When students struggle with certain subjects or need extra preparation for exams, many families turn to private tutoring as a way to fill the gaps. This practice of hiring tutors outside of regular school hours has become extremely common, as parents look for additional academic support to help their children perform better.

Private tutoring, commonly referred to as shadow education, has become a widespread practice as formal schooling systems often fail to meet the diverse learning needs of students [1]. This form of supplementary education has grown particularly fast in developing countries, where public school resources are limited and teacher-student ratios remain inadequate [2]. Research on shadow education as a global phenomenon confirms that it has expanded across all continents and income levels, becoming a parallel curriculum that operates alongside formal schooling [3].

Nepal's education system has made notable progress over the past decade. According to the National Population and Housing Census 2021, the overall literacy rate reached 76.3 percent among the population aged five years and above, up from 65.9 percent in 2011. The country has also experienced rapid urbanization during this period, with the urban population share jumping from 17.07 percent in 2011 to 66.17 percent in 2021 [4]. Despite these improvements, challenges within the mainstream education system persist. Overcrowded classrooms, inconsistent teaching quality, and limited resources in public schools continue to push families toward private tutoring. In cities like Kathmandu, dual-income households have become increasingly common, leaving working parents with limited time for direct academic supervision of their children. As a result, many urban families look for home tutors as a practical solution.

The phenomenon of shadow education is deeply rooted in Nepal's educational landscape. Thapa's study on private and public school competition in Nepal highlights how

supplementary tutoring has become an established practice, with families across income levels relying on it to fill gaps left by formal schooling [5]. Studies further indicate that approximately 68 percent of Grade 10 students in Nepal receive some form of private tutoring, confirming how widespread this practice has become at the secondary level.

Despite this growing demand, Nepal’s tutoring market remains fragmented and informal. Parents and students typically rely on word-of-mouth recommendations or unstructured social media groups to find tutors. These methods offer no verification of tutor credentials, no proximity-based filtering, and no intelligent matching. With smartphone penetration in Nepal reaching 65 to 71 percent, the digital infrastructure exists to address these problems through a dedicated mobile platform.

This proposal presents EdumentX, a location-based mobile application designed to modernize home tutor discovery in Kathmandu Valley. The platform enables students and parents to find nearby verified tutors through an interactive map, enroll in long-term monthly plans, and receive personalized recommendations through an AI-powered RAG chatbot.

1.2 Problem Statement

The process of finding a home tutor in Nepal is currently slow, unreliable, and frustrating for everyone involved. Both students seeking academic help and tutors looking for teaching opportunities face significant difficulties due to the lack of any organized system.

The core problems are:

- (i) There is no way for parents to check whether a tutor is actually qualified or trustworthy before hiring them. Most families rely on recommendations from people they know, which limits their options and gives no real measure of teaching quality or safety.
- (ii) Independent tutors, including university students looking for part-time work, retired teachers, and subject specialists, have no proper platform to advertise their services. Many end up going through agencies that take large commissions, while others simply cannot find students outside their own neighborhood.
- (iii) The tools people currently use to find tutors, mainly WhatsApp groups and Face-

book pages, are not designed for this purpose. Posts are unorganized, location details are vague (just area names with no actual distance), there is no way to filter or search, and there is no system to verify anyone's identity or qualifications.

At present, no mobile application exists in Nepal that lets users find tutors on a map, filter by distance, or get AI-based suggestions. EdumentX is designed to solve these problems by offering proximity-based search on an interactive map, a verification system where admins review tutor documents and award Blue Tick badges, a structured monthly enrollment process, and a RAG-powered chatbot that pulls live tutor data to give personalized recommendations.

1.3 Objectives

- To develop a location-based mobile application that enables students and parents to discover and connect with nearby home tutors in Kathmandu Valley using map-based proximity search.
- To integrate an AI-powered RAG chatbot that allows users to find suitable tutors through natural language queries grounded in live platform data.
- To implement a tutor verification system that helps parents make safe and informed hiring decisions.

1.4 Scope

EdumentX is a role-based mobile platform for connecting students and parents with home tutors based on location, subject, and monthly enrollment preferences. The scope of this project includes the following:

- **Target Users:** Students, parents, independent home tutors, and platform administrators.
- **Geographic Focus:** Kathmandu Valley, where smartphone penetration and internet connectivity are sufficient to support the platform.
- **Service Type:** Physical (in-person) home tuition services only; no online or video-based classes.

- **Academic Levels:** School and college students at SEE, +2, and undergraduate levels.
- **Platform:** Cross-platform mobile application for Android and iOS built with React Native (Expo).
- **Discovery Methods:** Map-based search with proximity ranking and an AI chatbot for natural language queries.
- **Enrollment Model:** Long-term monthly enrollment plans (1, 3, 6, or 12 months) managed through structured request and response flows.
- **Verification:** Blue Tick badge system for tutor credential verification by admin.
- **Group Learning:** Support for group tuition batches created by tutors for shared, cost-effective learning.

2. LITERATURE REVIEW

2.1 Related Work

This section reviews recent academic research on location-based tutor matching systems, AI-powered educational platforms, and identifies the gaps that EdumentX aims to address.

A 2022 study presented at ASCILITE introduced MENTOR, an intelligent mobile peer tutoring application designed for both face-to-face and remote tutoring [6]. The system focused on matching peer tutors with students based on subject expertise and availability. While it demonstrated the value of mobile platforms for tutoring, it did not include any location-based features such as map visualization or proximity ranking. The matching was based solely on academic parameters without considering geographic distance between users.

A 2026 study published in the Journal of Emerging Technologies and Innovative Research described a Flutter-based mobile application for finding local tutors in India [7]. The system used Google Maps API for proximity-based tutor search and Firebase as the backend database. Students could filter tutors by subject, availability, and location. The application calculated distances between users using the Haversine formula and included features for booking and rating tutors. However, the researchers noted that AI-based recommendations were left as future scope. The entire matching process relied on manual filtering by the user, with no intelligent ranking or conversational search capability.

In 2025, researchers from Malaysia developed iTeachU, a system that combined AI with geolocation for tutor matching [8]. The platform used content-based filtering along with the K-Nearest Neighbors algorithm, incorporating Haversine distance calculations to rank tutors by both preference match and physical proximity. It was built using Flutter for the frontend and Flask for the backend. While the system demonstrated effective ranking within the Melaka region, its AI component was entirely rule-based. Users could not interact with the system using natural language. Instead, they had to select from predefined filters and dropdown menus, making the experience less flexible than

a conversational interface.

All of these studies represent important steps toward technology-driven tutor discovery, but several gaps remain when considering the Nepali context. No existing research or platform provides geospatial visualization of tutors on interactive maps with service area coverage for Nepal. City-level dropdown menus, which are used by most platforms, are not useful in dense urban areas like Kathmandu where a tutor two kilometers away is very different from one ten kilometers away. Independent part-time tutors such as university students and retired teachers lack a proper digital channel to reach nearby students, as existing systems tend to favor agency-managed tuition centers. None of the reviewed systems offer a natural language interface where users can simply type what they need instead of navigating through multiple filter options. While some platforms claim AI-based matching, none combine proximity, qualifications, availability, and budget into a single intelligent ranking powered by a large language model. The current methods available in Nepal, primarily social media groups, require significant manual effort to browse posts, contact agencies, and compare options without any automated assistance.

EdumentX addresses these gaps by combining map-based visualization, an AI-powered conversational interface using RAG architecture for natural language queries, a structured verification system with Blue Tick badges, and a focus on empowering independent tutors to connect with nearby students efficiently.

2.2 Related Theory

2.2.1 Location-Based Services

Location-Based Services deliver context-aware services using a device’s real-time geographic position obtained through GPS, cellular triangulation, or Wi-Fi. In EdumentX, location-based services power the proximity-based tutor discovery and map visualization features. Tutor coordinates are stored as Firestore GeoPoint fields, which allow geospatial queries to be executed directly from the client application.

2.2.2 Haversine Distance Formula

The Haversine formula computes the great-circle distance between two points on Earth’s surface, accounting for the planet’s curvature. EdumentX uses this formula to rank tutors

by their true distance from the student. The computation proceeds as follows:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (2.1)$$

$$c = 2 \arctan 2(\sqrt{a}, \sqrt{1-a}), \quad d = R \cdot c \quad (2.2)$$

where ϕ_1, ϕ_2 are latitudes in radians, λ_1, λ_2 are longitudes in radians and $\Delta\phi$ and $\Delta\lambda$ are the coordinate differences, $R \approx 6,371$ km, and d is the output distance. Degree values from GeoPoints are converted using $\phi_{\text{rad}} = \phi_{\text{deg}} \times \frac{\pi}{180}$. For the 1 to 20 km range typical in Kathmandu Valley, this formula provides meter-level accuracy.

2.2.3 Geofencing and Radius-Based Filtering

Before applying the Haversine formula, EdumentX uses a rectangular bounding box pre-filter to narrow down the candidate set and reduce computation [9]. For a given search radius r and student position (ϕ_s, λ_s) :

$$\Delta\phi_{\max} = \frac{r}{R} \cdot \frac{180}{\pi}, \quad \Delta\lambda_{\max} = \frac{r}{R \cdot \cos(\phi_s)} \cdot \frac{180}{\pi} \quad (2.3)$$

Only tutors whose coordinates fall inside this bounding box proceed to the exact Haversine distance check, which keeps query response times low.

2.2.4 K-Nearest Neighbors Algorithm

KNN is a non-parametric algorithm used in EdumentX to rank tutors by overall suitability. Each tutor is represented as a feature vector $\mathbf{t} \in \mathbb{R}^5$:

$$\mathbf{t} = [d, r, q, e, \bar{v}] \quad (2.4)$$

where d is the Haversine distance, r is the monthly rate, q is the qualification score, e is years of experience, and $\bar{v}$ is the average rating. All features are scaled to the range $[0, 1]$ using min-max normalization: $x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$. The similarity between a student query vector $\mathbf{s}$ and a tutor vector $\mathbf{t}$ is measured by weighted Euclidean distance:

$$D(\mathbf{s}, \mathbf{t}) = \sqrt{\sum_{j=1}^n w_j \cdot (s_j - t_j)^2}, \quad \text{Top-}k = \arg \min_{\mathbf{t} \in \mathcal{T}} D(\mathbf{s}, \mathbf{t}) \quad (2.5)$$

The default value of $k = 5$. Feature weights are assigned based on their importance for home tuition selection:

Table 2.1: KNN Feature Weights for Tutor Recommendation

Feature	Symbol	Weight w_j
Geographic Distance	d	0.35
Monthly Rate	r	0.25
Qualification Score	q	0.20
Average Rating	$\bar{v}$	0.15
Experience (years)	e	0.05

2.2.5 Retrieval-Augmented Generation

Retrieval-Augmented Generation is an approach that combines information retrieval with text generation to produce responses grounded in actual data rather than relying solely on a language model’s training data. The original RAG framework was introduced by Lewis et al., who showed that combining a pre-trained retriever with a sequence-to-sequence generator significantly improves accuracy on knowledge intensive tasks [10]. In EdumentX, RAG is used to retrieve matching tutor profiles from Firestore based on the user’s query and inject them as context into the language model’s prompt, ensuring that recommendations are based on real, live platform data. A recent survey on RAG chatbots in education confirms that this approach reduces hallucination and improves the relevance of generated responses in educational contexts [11].

2.2.6 React Native and Expo

React Native enables building native iOS and Android applications from a single JavaScript and React codebase. Expo adds a managed toolchain for accessing device features such as location, camera, and push notifications. It also supports over-the-air updates, which reduces development and release cycle time. EdumentX uses Expo SDK 54 with the New Architecture (Fabric/TurboModules) enabled.

2.2.7 Firebase Platform and Supabase Storage

Firebase is Google’s Backend-as-a-Service suite. Four services form the primary EdumentX backend: **Authentication** for email/password and Google sign-in; **Firestore** for real-time NoSQL storage with native GeoPoint fields and Security Rules; **Supabase Storage** (1 GB free tier, keyless) for verification documents and demo videos; and the

Firestore Admin SDK (Node.js, used out-of-band) for bootstrapping the administrator role.

EdumentX does *not* depend on Cloud Functions: the client computes Haversine distance, KNN ranking, and the retrieval step for the RAG pipeline directly. This keeps the project within the free Spark tier and avoids the paid Blaze plan that the project team cannot sign up for.

2.2.8 OpenStreetMap and Open Geocoders

To avoid the paid Google Maps Platform API, EdumentX uses OpenStreetMap raster tiles rendered through `react-native-maps`' `<UrlTile>` component (keyless) and the keyless Nominatim service for forward geocoding. Distance and KNN matching are computed client-side, which is feasible at the Kathmandu-Valley scale (target database size is in the low thousands of tutors).

3. PROPOSED METHODOLOGY

3.1 Software Development Model

The EdumentX project follows the Agile Iterative-Incremental development model. The system is built through short development cycles called sprints, where each sprint delivers a working piece of the application. After each sprint, the output is reviewed, tested, and improved before moving to the next cycle. This model suits EdumentX because the project has distinct modules like map search, AI chatbot, and enrollment that can be developed independently. The RAG chatbot requires experimentation with the language-model endpoint, and an iterative approach allows the team to refine its accuracy over multiple cycles. The 14-week academic timeline is divided into seven 2-week sprints with clear deliverables at each stage. Figure 3.1 shows the phases of the iterative-incremental model.

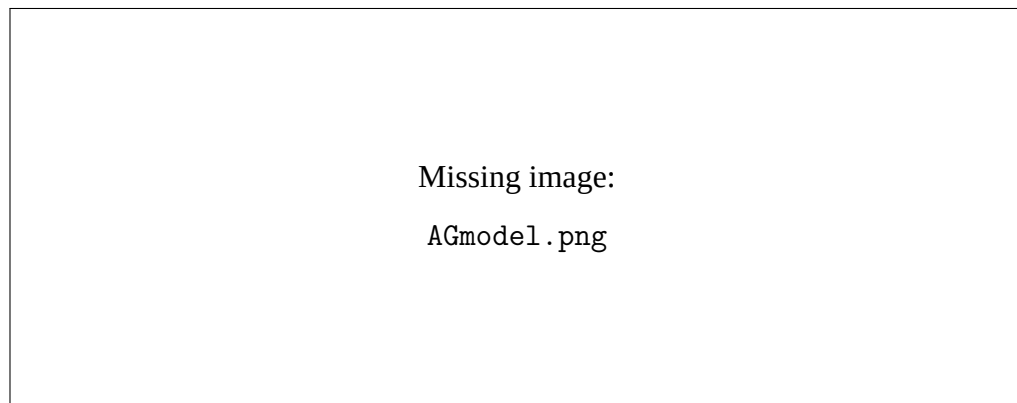


Figure 3.1: Phases of Iterative-Incremental Model

Source: GeeksforGeeks

3.2 Feasibility Study

The feasibility study evaluates whether the proposed system can be realistically developed and deployed under existing constraints. The project follows standard SDLC phases from planning and analysis through implementation, testing, and deployment.

3.2.1 Technical Feasibility

The proposed system is technically feasible given the team's proficiency and access to essential tools and technologies.

Technology Stack:

- **Frontend:** React Native (Expo SDK 54) for cross-platform mobile development.
- **Backend/Database:** Firebase (Authentication, Firestore) and Supabase Storage.
- **Map Integration:** OpenStreetMap tiles via `react-native-maps`' `<UrlTile>`.
- **Geocoding:** Nominatim (keyless).
- **AI Integration:** Free-tier LLM endpoint (Groq or HuggingFace Serverless Inference) accessed directly from the client.
- **State Management:** Zustand + React Context.

The development team has foundational skills in frontend and backend development, mobile application programming, database management, and basic AI integration. Development tools include VS Code, Git/GitHub, Postman, and Firebase Console.

3.2.2 Economic Feasibility

The proposed system is economically feasible as it uses free open-source tools and cloud services with generous free tiers. The Firebase Spark Plan eliminates the need for a custom server. Cost assumptions are aligned with official Firebase pricing documentation.

Table 3.1: Economic Feasibility Estimation

Item	Estimated Cost
Domain (.com.np)	Rs. 0 (free via register.com.np)
Hosting (Firebase Spark Plan)	Free tier
Firestore reads/writes/storage	Free tier (Spark limits)
Supabase Storage	Free tier (1 GB)
OpenStreetMap tiles	Free (keyless)
Nominatim geocoding	Free (keyless, rate-limited)
LLM endpoint (Groq / HuggingFace)	Free dev tier
Development Tools	Free (open source)
Miscellaneous	Rs. 1,000–2,000
Total Initial Cost	Rs. 1,000–2,000

For academic demonstration purposes, free tiers and student credits will suffice.

3.2.3 Operational Feasibility

The system is operationally feasible as it is accessible on both Android and iOS platforms. The interface is designed to be intuitive for parents, students, and tutors. Core requirements such as internet access and basic device compatibility are easily met by the target audience in urban Nepal. The platform requires minimal training for users.

3.2.4 Legal and Ethical Feasibility

- All user data is collected with consent and used only for platform functionality and demonstration.
- No real personal data will be published externally; dummy data will be used for academic presentations.
- Location information is used solely for proximity searches and is not shared with third parties.
- As an academic project, it is not intended for immediate commercial use.
- Exact locations of tutors or students are not displayed publicly; only approximate area coverage is shown.

3.2.5 Scalability

- Firebase’s cloud infrastructure scales automatically as user base grows.
- Additional cities can be added without major architectural changes.
- Future enhancements can include in-app payments, video calling, and analytics dashboards.

3.3 Requirement Analysis

3.3.1 Functional Requirements

R.1: User Registration and Authentication

- Users can register using email and password, or sign in with Google.
- Profile creation includes name, phone, email, location, and role selection (student or tutor).
- Users can update their profile information at any time.

R.2: Tutor Profile Management and Verification

- Tutors can create profiles with qualifications, subjects, experience, monthly rate, availability, service radius, and maximum student capacity.
- Tutors can upload citizenship ID, academic certificates, and a one-minute teaching demo video for verification.
- Admin reviews documents and grants Blue Tick badges.
- Verified tutors get higher ranking in search results and chatbot recommendations.
- Verified tutors can create group batch offerings that students can browse and join.

R.3: Tutor Search, Filtering, and Location-Based Discovery

- Students can search tutors by subject, proximity, price range, qualification, and verification status.
- Nearby tutors are displayed on an interactive map with service area markers.

- Distance is calculated using the Haversine formula with radius-based filtering.
- Results can be sorted by distance, rating, price, or experience and filtered by availability.

R.4: AI Chatbot Assistant

- Users can type natural language queries to find tutors.
- The chatbot extracts parameters like subject, location, budget, and plan duration from the query.
- It returns personalized recommendations from live Firestore data using RAG architecture.
- Multi-turn conversations are supported for follow-up queries.

R.5: Enrollment and Monthly Plan System

- Students send enrollment requests specifying plan duration, start date, days per week, and teaching location.
- Tutors can accept, reject, or counter-offer through their dashboard.
- Active enrollments include an optional trial week and automatic monthly renewal reminders.
- The system prevents enrollment when a tutor is at full capacity.

R.6: Rating and Review System

- Students can rate tutors on a 1 to 5 star scale and write reviews after an enrollment period.
- Only students with active or completed enrollments can leave reviews.
- Average ratings are displayed on tutor profiles.

R.7: Admin Panel

- Admin dashboard for reviewing tutor verification documents and granting badges.
- Ability to approve, suspend, or remove user accounts.
- View platform statistics, enrollment counts, and user feedback.

3.3.2 Non-Functional Requirements

R.1: Performance

- The application should load within 5 seconds on a normal connection.
- Search results and map rendering should complete within 3 seconds.
- The system should handle concurrent users without noticeable slowdown.

R.2: Security

- Passwords are hashed by Firebase Authentication; no personal data is stored in plain text.
- All communications use HTTPS encryption.
- Protection against XSS, CSRF, and Firestore rule-bypass attacks via Security Rules.
- LLM API keys (when used) are kept in environment-scoped configuration, not in the client bundle.

R.3: Usability

- The interface should be clean and intuitive for both technical and non-technical users.
- Navigation should require minimal clicks with clear error messages.

R.4: Availability and Compatibility

- The application should be available 24/7 with at least 90 percent uptime.
- It should work on Android 8.0+ and iOS 12.0+ with responsive design for different screen sizes.
- Network switching between Wi-Fi and mobile data should not cause crashes.

R.5: Scalability

- The database and cloud services should support scaling as the user base grows.
- The code structure should allow adding new features without major changes.

3.4 Proposed System Design

3.4.1 System Architecture

Figure 3.2 shows the overall system architecture of EdumentX. The architecture has three layers: Client Side, Serverless Backend, and External Services.



Figure 3.2: System Architecture of EdumentX

The **Client Side** is the React Native mobile application built with Expo. It serves as the single entry point for all users and communicates with the backend using the native Firebase SDK (`@react-native-firebase/*`).

The **Serverless Backend** runs entirely on Firebase with three services. **Firestore Authentication** handles user registration and email/password plus Google sign-in. **Cloud Firestore** stores all structured data including user profiles, tutor profiles, enrollment requests, and reviews, with tutor locations stored as GeoPoint fields and protected by Security Rules. **Supabase Storage** (keyless, 1 GB free tier) holds tutor verification documents and demo videos.

The **External Services** layer includes OpenStreetMap tiles (rendered via `<UrlTile>`) for map rendering, Nominatim for keyless forward geocoding, and a free-tier LLM endpoint (Groq or HuggingFace Serverless Inference) for the RAG chatbot. Importantly, EdumentX does *not* depend on Cloud Functions, Google Maps SDK, or any paid LLM — every external call is either keyless or fits within a free quota.

3.4.2 Use Case Diagram

Figure 3.3 shows the use case diagram with three actors: Students/Parents, Tutors, and Admin.



Figure 3.3: Use Case Diagram

Students and parents can register, turn on location, search for tutors using filters or the AI chatbot, view tutor profiles, send enrollment requests, and rate tutors after completing an enrollment.

Tutors can register, set their service area, upload documents for verification, manage their profile and availability, respond to enrollment requests, create group batches, and manage student capacity.

Admin can log in to the admin panel, verify tutor credentials, manage user accounts, and grant Blue Tick badges.

3.4.3 Data Flow Diagram

Figure 3.4 shows the Level 0 Data Flow Diagram. Users interact with the EdumentX system by sending requests and receiving responses. The system processes these requests and stores or retrieves data from Firebase Firestore.



Figure 3.4: Level 0 Data Flow Diagram

Figure 3.5 shows the Level 1 Data Flow Diagram with detailed system processes

including authentication, tutor search, AI chatbot recommendation, enrollment management, tutor verification, and rating handling.



Figure 3.5: Level 1 Data Flow Diagram

3.4.4 RAG Chatbot Workflow

Figure 3.6 illustrates the RAG workflow used by the EdumentX AI chatbot.

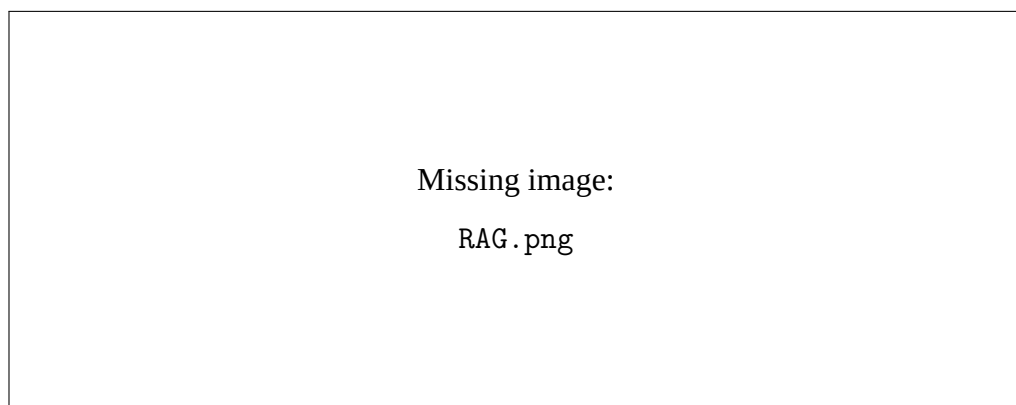


Figure 3.6: RAG-Based AI Chatbot Workflow

Source: freeCodeCamp

The chatbot works through a six-step pipeline. First, the student types a natural language query like “I need a science tutor near Lalitpur for 3 days a week under Rs 5,000.” The full conversation history is included so the chatbot can handle follow-up questions. Second, the client extracts structured parameters from the query — subject, location, budget, days per week, plan duration — using a keyless LLM endpoint. Third, the client queries the Firestore tutors collection, applies the bounding-box pre-filter followed by exact Haversine distance, and applies budget and subject filters. Fourth, the top ten matching tutor profiles are formatted and injected into the LLM’s prompt as

context data — this is the retrieval-augmented step. Fifth, the language model generates a response recommending three to five tutors with name, rate, distance, rating, and verification status. Sixth, the response is rendered as a chat message in the app. Identical queries are cached for 15 minutes to keep the free-tier token budget healthy.

3.5 Implementation Details

3.5.1 Frontend Development

The frontend is built using React Native with Expo SDK 54 and the New Architecture (Fabric/TurboModules) enabled. The styling system is NativeWind 4.2.x (a Tailwind compiler for React Native) with a custom “Night & Sand” premium design palette (slate-night background, sand surface, amber accent) defined in `tailwind.config.js`. The codebase uses standard React Native primitives (`View`, `Text`, `Pressable`, `TextInput`, `ScrollView`) and avoids inline style objects. State is managed with Zustand for global session state and React context for screen-local state. Navigation uses `expo-router` on top of React Navigation.

Key screens implemented at the mid-term milestone include authentication (email + password, Google Sign-In), role selection, student and tutor profile onboarding, student and tutor home dashboards, profile parity screens (with avatar upload), and an admin surface (verification queue, statistics, user management, admin profile setup).

3.5.2 Backend Development

The backend uses a pure Firebase stack with no custom server. Firebase Authentication handles email/password and Google sign-in and session management. Firestore stores all application data with tutor locations as GeoPoint fields, and Security Rules control read/write access per role. Supabase Storage (1 GB free, no card) holds verification documents and demo videos. Composite indexes on subject, location, and rate fields enable efficient multi-criteria searches. The Firebase Admin SDK is used out-of-band from a local Node.js script (`scripts/seedAdmin.ts`) to grant the administrator role to a fixed allow-list of email addresses, since the `admins` collection is locked to Admin-SDK writes only.

3.5.3 Testing Strategy

Unit testing is performed with Jest for individual React Native components and pure functions (Haversine, KNN, RAG cache). Integration testing covers Firebase Authentication flows, Firestore Security Rules, and the upload-to-Supabase pipeline. User acceptance testing is being conducted with a small group of student and tutor test accounts. Performance testing focuses on cold-start time of the Expo dev client, map-tile render latency, and the end-to-end RAG query latency at typical Kathmandu-Valley data sizes (tens to low thousands of tutors).

4. TASK ACCOMPLISHED

This chapter summarizes the work completed as of the mid-term defense. EdumentX is being built as a sequence of numbered *stages*; each stage is independently demonstrable. Stages 0 through B are complete and ready for committee review. Stages C onward are listed in Chapter 5.

4.1 Stage 0 — Project Setup and Documentation Scaffolding

- Repository initialized with Expo SDK 54, TypeScript, React Native, and expo-router file-based routing.
- Documentation/ folder hierarchy established (00-Overview, 01-Architecture, 02-Design-System, 03-Implementation-Guides, 04-Firebase, 05-Build-and-Deploy, 06-Prompts, 07-Design-Context, 97-Educational-Contents, 98-Reference-BasoBas, 99-Archive).
- LOG.md created and used as the team-wide change log.
- EdumentX_Master_Context_State.md established as the single source-of-truth status document.

4.2 Stage 1 — Design System: “Night & Sand” Premium Palette

- Design tokens registered in `tailwind.config.js` covering surface, text, border, and brand colors (night, sand, amber, etc.).
- `constants/colors.ts` provides raw hex values for the narrow set of SVG illustrations that need them.
- Shared primitives (Button, Card, Screen, Header) extracted to `components/`.

4.3 Stage 1.5 — NativeWind Migration (Tamagui Removal)

- Removed all `@tamagui/*` packages and the `tamagui.config.ts` file.
- Migrated the entire codebase from Tamagui styled components to standard React Native primitives with NativeWind `className` props.
- Replaced the Tamagui theme provider with a plain NativeWind setup driven by the `tailwind.config.js` tokens.

4.4 Stage 2 — Native Firebase Auth (RNFirestore Pivot)

- Adopted the native `@react-native-firebase/*` SDK packages (modular API: `getApp`, `getAuth`, `getFirestore`, `createUserWithEmailAndPassword`, `signInWithCredential`).
- `google-services.json` shipped at the native build time; no `process.env` Firebase config.
- Implemented two sign-in methods on `screens/auth/EmailSignUp.tsx`: Email + Password and Google Sign-In.
- Auth Source-of-Truth routing lives in `app/_layout.tsx` as a 5-step decision tree: unauthenticated $\rightarrow$ `/email-signup`; authenticated but unverified and password-provider $\rightarrow$ the “check your inbox” pending panel; verified but role-less $\rightarrow$ `/role-selection`; verified with role $\rightarrow$ matching dashboard.
- Resolved the “Amnesia Login Loop” bug (existing user was being sent to `/role-selection` on every sign-in) by reading `users/{uid}.role` inside the `onAuthStateChanged` callback and writing it to the Zustand store before the redirect effect runs.

4.5 Stage 2b — Onboarding and Profile Subcollection Writes

- `screens/auth/RoleSelection.tsx` writes `role` to `users/{uid}` and routes to `/profile-student` or `/profile-tutor`.

- `screens/auth/StudentProfileScreen.tsx` and `screens/auth/TutorProfileScreen.tsx` use `writeBatch` to atomically write both the parent `users/{uid}` doc and the `users/{uid}/<role>Profile/default` subcollection doc, with `photoUrl` included as an optional field once `AvatarUploader` succeeds.
- Onboarding collects: name, phone, location (city-level Nominatim geocode), and role-specific fields (student: grade and subjects; tutor: headline, bio, monthly rate, subjects, years of experience).
- `LocationField` enforces `MIN_CITY_LENGTH = 3` for the city-field validation rule.

4.6 Stage 3 — Dashboards and Live Firestore Reads

- `screens/student/student_home.tsx` subscribes to the student's `users/{uid}` and `users/{uid}/studentProfile/default` docs via `onSnapshot` and shows the real `fullName` and `locationLabel`.
- `screens/tutor/tutor_home.tsx` does the same for tutors and surfaces the `verified` flag.
- `MOCK_TUTORS` list in `student_home.tsx` is a placeholder for the tutors collection query, which will be wired in Stage C.
- Tutor metric values (rating, reviews, response rate, monthly earnings) are still mock data in `tutor_home.tsx`; the live aggregation is queued for Stage C.

4.7 Stage A — Auth UX Polish, Bottom Nav, and Parity Foundations

- “Sign up” / “Log in” toggle on `EmailSignUp.tsx` with a “check your inbox” panel after signup.
- The “**I’ve verified — continue**” button is the only place the app calls `auth.currentUser.reload()` without it, the cached `User.emailVerified` flag stays stale and the layout guard refuses to advance.

- Fixed the bottom-nav routing so that the tutor “Profile” tab now lands on `EditTutorProfile` instead of looping back to the dashboard.
- Real `fullName` and `locationLabel/verified-flag` data wired into the dashboard heroes and the profile view.
- CTA buttons constrained to a max width so they do not stretch across the full screen on tablets.
- Admin surface scaffolded: `AdminHome`, `Statistics`, `VerificationQueue`, `UserManagement`, and `AdminProfileSetup`.
- Administrator access is granted out-of-band by writing `admins/{uid}` from a local firebase-admin seed script (`scripts/seedAdmin.ts`) because the `admins` collection is locked to Admin-SDK writes only.

4.8 Stage A.5 — Tutor Profile Parity, Avatar Persistence, and Logout

- `screens/tutor/edit_profile.tsx` rewritten as a parity screen modeled on `StudentProfile.tsx`: slate hero, identity card with avatar bubble, name, email and a Change-photo pill, two `EditableField` rows (Headline + About me), locked email row, three `MenuRows`, destructive Log out button, and a version footer.
- `MenuRow` extracted from `StudentProfile.tsx` into `components/forms/MenuRow.tsx` so both screens import the same primitive.
- `photoUrl` persistence added in three places: `TutorProfileScreen.tsx` and `StudentProfileScreen.tsx` include `photoUrl: avatarUri` in the onboarding `writeBatch` payload; the new `edit_profile.tsx` parity screen and the `StudentProfile.tsx` screen both auto-persist `photoUrl` to Firestore via a `useEffect` on local-state change so the avatar survives a reload.
- Tutor profile tab now has a working destructive Log out button. The previously unused `handleSignOut` in `tutor_home.tsx` was removed (clearing an eslint warning).

- Upload pipeline replaces the “pickImage → Coming soon” alert with a real `uploadAvatar` call to Supabase Storage’s `public-avatars/{uid}.jpg` bucket with a cache-busting query parameter.

4.9 Stage B — 3D Onboarding Polish

- Onboarding slide 1 (DiscoverScene3D) and slide 2 (AiOrb3D) re-rendered with the new “Night & Sand” palette.
- The 3D renderer is loaded from `react-three-fiber/native` only (never the default `react-three-fiber` entry, which pulls DOM-only Web dependencies into the bundle).
- Metro config extended to support `react-three-fiber/native` and the asset pipeline for the 3D scenes.

4.10 Verification Summary

- `npm run typecheck` (TypeScript strict mode, `tsc --noEmit`) returns **0 errors**.
- `npm run lint` returns **0 errors** and a small, audited set of warnings.
- `firebase deploy --only firestore:rules` applies the locked-down admins collection rules so the seed script can grant admin access.
- Manual end-to-end test on Android: sign up as student, complete onboarding, see real name in dashboard, change avatar, log out, sign back in as tutor, complete tutor onboarding, see real headline on profile tab, log out, sign in as admin (after running `npm run seed:admin`), complete admin profile setup, land on the admin dashboard.

5. TASKS REMAINING

The work listed below remains to be done between the mid-term defense and the final minor project defense. The team will incrementally update this chapter as stages C through J are completed.

5.1 Stage C — Live Tutor Query and KNN Ranking

- Replace the `MOCK_TUTORS` list in `student_home.tsx` with a real-time `onSnapshot` query against the `tutors` collection.
- Implement the Haversine + KNN ranking pipeline client-side (already documented in §2.2.2 and §2.2.4).
- Surface the live ranking in the student search screen (distance, monthly rate, qualification, rating).
- Wire the four tutor dashboard metric cards (rating, reviews, response rate, monthly earnings) to real aggregations.

5.2 Stage D — RAG Chatbot End-to-End

- Implement the six-step RAG pipeline described in §3.4 with a free-tier LLM endpoint (Groq or HuggingFace Serverless Inference).
- Add a 15-minute query-result cache in client storage to keep token usage within the free quota.
- Support multi-turn conversations by sending the full chat history with each new query.
- Render the chatbot UI (chat bubbles, message input, loading state) inside the student and tutor home screens.

5.3 Stage E — Map Search and Service-Area Visualization

- Integrate OpenStreetMap raster tiles via `react-native-maps`' `<UrlTile>` component.
- Plot a marker for each tutor on the map using the `GeoPoint` stored in the tutor's profile subdoc.
- Draw the tutor's service-radius circle and centre the map on the student's current location (or the city they typed during onboarding).
- Wire marker taps to a bottom sheet showing the tutor's name, rate, distance, and Blue Tick badge.

5.4 Stage F — Verification Queue and Blue Tick Pipeline

- Build the tutor document-upload screen (citizenship ID, academic certificates, 1-minute teaching demo).
- Build the admin verification queue (`screens/admin/VerificationQueue.tsx`) with four sections: New Tutor Verifications, Pending Edits, Under Review (Info Requested), Decided.
- Persist verification state on `users/{uid}/tutorProfile/default.verified` and surface the Blue Tick badge in search results and the RAG chatbot's response.

5.5 Stage G — Enrollment Request and Response Flow

- Build the student-side enrollment request form (plan duration 1/3/6/12 months, start date, days per week, location).
- Build the tutor-side response dashboard (accept / reject / counter-offer).
- Enforce tutor capacity on the server side via Firestore Security Rules.
- Add the optional trial week and monthly renewal reminder flows.

5.6 Stage H — Rating and Review System

- Allow students to rate tutors on a 1–5 star scale and write a review after an enrollment period.
- Restrict the write to the rating subcollection via a Firestore Security Rule that checks `enrollments/{id}.studentUid == request.auth.uid` and `status == completed`.
- Surface the average rating on the tutor’s public profile and as a feature vector component for the KNN ranking.

5.7 Stage I — Admin Statistics, User Management, and Polish

- Replace the mock-data KPI grid + weekly enrollment + subject demand bar lists in `Statistics.tsx` with real Firestore aggregations.
- Wire the `UserManagement` search, filter, suspend/reinstate, and soft-delete flows end-to-end (the buttons exist; the writes are pending).
- Finalize the admin profile setup screen so the admin’s display name appears across Home, Statistics, and Profile heroes.

5.8 Stage J — Final Hardening, Performance Pass, and Demo

- End-to-end test pass on Android and iOS against a seeded Firestore instance (50 tutors, 10 students, 2 admins, full enrollment / review / verification flow).
- Cold-start time < 5 s; map-tile render < 3 s; RAG end-to-end < 4 s on Kathmandu-Valley data sizes.
- Production-grade Android and iOS dev clients via EAS Build.
- Final report write-up and demo rehearsal.

REFERENCES

- [1] M. Bray. *The Shadow Education System: Private Tutoring and Its Implications for Planners*. Fundamentals of educational planning. UNESCO, International Institute for Educational Planning, Paris, 2007.
- [2] M. Bray and C. Lykins. *Shadow Education: Private Supplementary Tutoring and Its Implications for Policy Makers in Asia*. CERC monograph series in comparative and international education and development. Asian Development Bank, Manila, 2012.
- [3] Y. C. Kim and J. H. Jung. Shadow education as worldwide curriculum studies. *Curriculum Studies Worldwide*, 1(1):1–15, April 2019.
- [4] Central Bureau of Statistics. National population and housing census 2021: National report, 2023.
- [5] Amrit Thapa. Does private school competition improve public school performance? the case of nepal. *International Journal of Educational Development*, 33(4):358–366, 2013.
- [6] Sheng-Hung Chung and Seng Chee Tan. Mentor – intelligent mobile online peer tutoring application for face-to-face and remote peer tutoring. *ASCILITE Publications*, pages 386–391, September 2022.
- [7] R. K. Sharma, A. Patel, and M. Singh. Local tutor finder application using flutter. *Journal of Emerging Technologies and Innovative Research (JETIR)*, 13(2):267–275, February 2026.
- [8] N. Shari, F. Ahmad, and R. Hassan. iteachu: Ai-driven tutor matching with geolocation integration. *International Journal of Research in Science and Innovation (IJRSI)*, 12(3):145–158, March 2025.
- [9] Yury Shevchenko and Ulf-Dietrich Reips. Geofencing in location-based behavioral research: Methodology, challenges, and implementation. *Behavior Research Methods*, 56, August 2023.

- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [11] Jakub Swacha and Michał Gracel. Retrieval-augmented generation (rag) chatbots for education: A survey of applications. *Applied Sciences*, 15(8):4234, 2025.